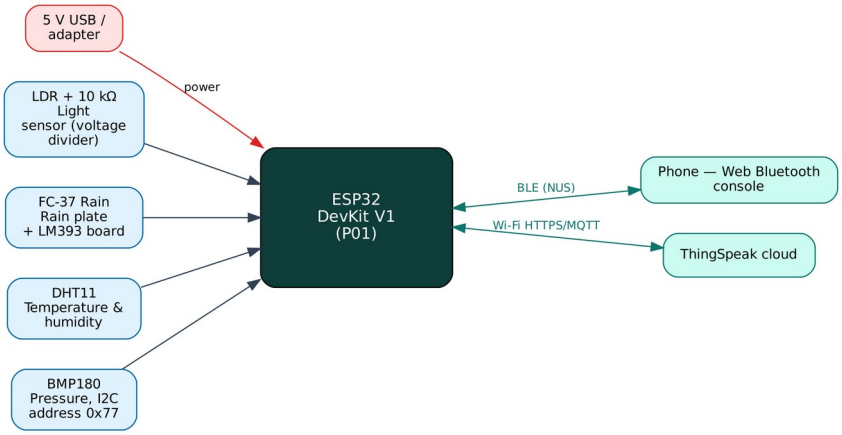
A PROJECT REPORT ON

# Smart Weather Station with Rain Forecast

IoT project using ESP32 with Bluetooth Low Energy and Wi-Fi (ThingSpeak)




**Project:** P01 · Intermediate level

**Peripherals:** DHT11, BMP180, LDR, FC-37 rain sensor

**Submitted by:** _____

**Under the guidance of:** _____

**Department / Institution:** _____

October 2026

# Certificate

This is to certify that the project report entitled **"Smart Weather Station with Rain Forecast"** is a bonafide record of the work carried out by _____ (Roll No. _____) in partial fulfilment of the requirements for the award of _____ in _____ during the academic year _____.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

# Acknowledgement

I express my sincere gratitude to my project guide _____ for continuous guidance and support, to the Head of the Department _____, and to all faculty members and friends who helped me complete this project successfully.

# Contents

**Abstract** ..... 6

**1. Introduction** ..... 7

1.1 Problem Statement ..... 7
1.2 Objectives ..... 7
1.3 Scope of the Project ..... 7
1.4 Organisation of the Report ..... 7

**2. Existing System and Proposed System** ..... 8

2.1 Existing Systems ..... 8
2.2 Proposed System ..... 8
2.3 Why Both BLE and Wi-Fi? ..... 8

**3. System Design** ..... 9

3.1 Block Diagram ..... 9
3.2 System Architecture ..... 9
3.3 Hardware Components and Cost ..... 9
3.4 ESP32 DevKit V1 ..... 10
3.5 Working Principle of Components ..... 10

**4. Circuit Design** ..... 11

4.1 Circuit Diagram ..... 11
4.2 Pin Connections ..... 11
4.3 Interfacing and Power Notes ..... 12

**5. Data Flow Diagrams** ..... 13

5.1 DFD Level 0 (Context Diagram) ..... 13
5.2 DFD Level 1 ..... 14
5.3 Data Dictionary ..... 15

**6. Software Design and Implementation** ..... 16

6.1 Development Environment ..... 16
6.2 Firmware Architecture ..... 16
6.3 Working of the System ..... 16
6.4 Flowchart ..... 17
6.5 BLE Communication Protocol ..... 18
6.6 Cloud Integration (ThingSpeak) ..... 18
6.7 Configurable Settings ..... 19

**7. Setup and Deployment Guide** ..... 20

7.1 Hardware Assembly ..... 20
7.2 Flashing the Firmware ..... 20
7.3 Cloud Setup (ThingSpeak) ..... 20
7.4 BLE App Setup (Web Bluetooth Console) ..... 21
7.5 First-Run Checklist ..... 22

**8. Testing and Results** ..... 23

8.1 Expected Output ..... 23
8.2 Troubleshooting ..... 23

**9. Discussion** ..... 24

9.1 Advantages ..... 24
9.2 Limitations ..... 24
9.3 Safety Precautions ..... 24
9.4 Security Considerations ..... 24
9.5 Future Scope ..... 24

**10. Conclusion..... 25**

**References..... 26**

**Appendix A: Project Source Code..... 27**

**Appendix B: Shared Library Code..... 31**

# Abstract

This project builds a low-cost, connected weather station around the ESP32 microcontroller. A DHT11 sensor measures air temperature and relative humidity, a BMP180 barometer measures atmospheric pressure, a light-dependent resistor (LDR) measures ambient light and an FC-37 rain plate detects rainfall. The ESP32 derives heat index and dew point, keeps a three-hour history of pressure and turns the pressure trend into a short-term forecast, using rules similar to the classical Zambretti forecaster. Readings are uploaded every 20 seconds over Wi-Fi to ThingSpeak, a free MATLAB-backed IoT cloud that stores the data and draws charts. At the same time the station acts as a Bluetooth Low Energy (BLE) peripheral, so a phone can show live values, change settings and configure Wi-Fi without any internet connection. The complete bill of materials costs well under ₹1,000.

**Keywords:** ESP32, Internet of Things, Bluetooth Low Energy, Wi-Fi, ThingSpeak, DHT11, BMP180, LDR, FC-37 rain sensor.

# 1. Introduction

The Internet of Things (IoT) connects physical devices — sensors and actuators — to networks so that data can be collected, analysed and acted on automatically. The ESP32 from Espressif Systems has made IoT affordable: a single ₹350 board combines a dual-core 240 MHz processor, Wi-Fi and Bluetooth Low Energy. This project uses both radios at once: BLE for direct, local interaction with a phone and Wi-Fi for cloud connectivity through ThingSpeak, a free IoT/cloud platform.

## 1.1 Problem Statement

Local weather can differ a lot from the city-level forecast shown by phone apps, which matters for farmers, gardeners, schools and small businesses. Commercial weather stations with data logging and online access cost several thousand rupees and use closed apps. There is a need for an affordable, open station that measures the local microclimate, keeps a history, gives a simple rain warning and can be viewed both on site and remotely.

## 1.2 Objectives

1. Measure temperature, humidity, pressure, light intensity and rain with low-cost sensors.
2. Compute derived values: heat index, dew point and a 3-hour pressure-trend forecast.
3. Publish all values to a free cloud dashboard (ThingSpeak) for remote monitoring and history.
4. Provide a BLE interface for live data, Wi-Fi provisioning and configuration without internet.
5. Raise rain-start and rain-stop events immediately on both channels.
6. Keep the total hardware cost below ₹1,000 using parts available in the Indian market.

## 1.3 Scope of the Project

The station is intended for homes, school labs, terraces, greenhouses and farms. It covers indoor and sheltered outdoor use; for permanent outdoor use the electronics must be placed in a ventilated radiation shield (Stevenson screen) with only the rain plate exposed. Wind speed and direction are outside the scope of this version.

## 1.4 Organisation of the Report

Chapter 2 reviews existing approaches and describes the proposed system. Chapter 3 covers system design and components, Chapter 4 the circuit, Chapter 5 the data flow diagrams, Chapter 6 the software, Chapter 7 the step-by-step setup of hardware, cloud and app, Chapter 8 testing, Chapter 9 discussion and Chapter 10 the conclusion. Complete source code is given in the appendices.

# 2. Existing System and Proposed System

## 2.1 Existing Systems

Typical hobby weather stations either show values only on a local LCD or upload to the cloud without any local access, so they become useless when the internet is down and need the Wi-Fi password to be hard-coded into the firmware. Most also report raw readings only.

## 2.2 Proposed System

The proposed system improves on these in the following ways:

- Dual connectivity: BLE for local access and setup, Wi-Fi for cloud logging; each works without the other.
- Wi-Fi credentials and the ThingSpeak key are entered from the phone and stored in flash, not in source code.
- Value-added data: heat index, dew point and a pressure-trend forecast instead of raw numbers only.
- Event-driven upload: an immediate upload when rain starts or stops, in addition to the periodic one.

## 2.3 Why Both BLE and Wi-Fi?

Wi-Fi gives the device internet access for remote monitoring, history and control from anywhere, but it depends on a router and an internet connection, and the device needs the Wi-Fi password before it can connect. BLE needs no infrastructure: any modern phone can connect directly within about 10-30 m. Using both gives:

- **Provisioning** — Wi-Fi credentials and cloud keys are sent from the phone over BLE and stored in flash; nothing secret is hard-coded or uploaded to GitHub.
- **Resilience** — local monitoring and control keep working when the internet or router is down.
- **Remote access** — the cloud dashboard works from anywhere when the internet is available.
- **Low cost** — no extra hardware; both radios are built into the ESP32.

The ESP32 shares one 2.4 GHz radio between Wi-Fi and Bluetooth by time-division coexistence. For this to work the Wi-Fi modem-sleep mode must stay enabled, which the firmware ensures. NimBLE, a lightweight BLE stack, is used instead of the default Bluedroid stack to leave more RAM for TLS (HTTPS/MQTTS) connections.

# 3. System Design

## 3.1 Block Diagram

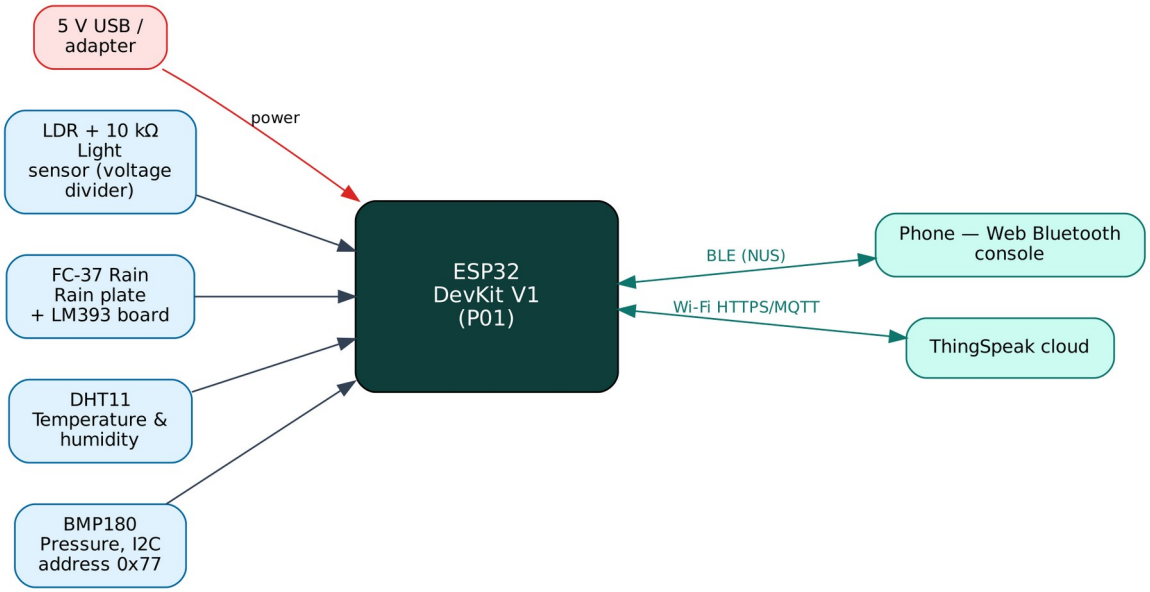



*Figure 1: Block diagram of the Smart Weather Station*

The ESP32 is the central controller. Input devices (blue) feed measurements to it, output devices (amber) are driven by it, and two communication links (green) connect it to the user's phone over BLE and to the ThingSpeak cloud over Wi-Fi. The board is powered from 5 V through its USB connector or VIN pin.

## 3.2 System Architecture

The system follows the standard three-layer IoT architecture:

- **Perception layer** — DHT11, BMP180, LDR, FC-37 rain sensor connected to the ESP32 GPIO, ADC, I2C, SPI or UART.
- **Network layer** — BLE 4.2 GATT server (Nordic UART Service) and Wi-Fi 802.11 b/g/n with TLS-encrypted HTTPS REST to ThingSpeak.
- **Application layer** — the Web Bluetooth console on the phone and the ThingSpeak dashboard / database.

## 3.3 Hardware Components and Cost

| # | Component | Qty | Specification | Cost |
|---|---|---|---|---|
| 1 | ESP32 DevKit V1 (ESP-WROOM-32, 30-pin) | 1 | Dual-core 240 MHz, Wi-Fi + BLE, 4 MB flash | ₹350 |
| 2 | Breadboard 830-point + jumper wires (M-M, M-F) | 1 | Prototyping and connections | ₹150 |
| 3 | Micro-USB data cable | 1 | Power + programming | ₹60 |
| 4 | DHT11 temperature & humidity module | 1 | 0-50 °C ±2 °C, 20-90 % RH ±5 % | ₹60 |
| 5 | BMP180 barometric pressure module (GY-68) | 1 | 300-1100 hPa, I2C, 3.3 V | ₹80 |

| # | Component | Qty | Specification | Cost |
|---|---|---|---|---|
| 6 | LDR 5 mm + 10 kΩ resistor | 1 | Light-dependent resistor in a divider | ₹10 |
| 7 | FC-37 / YL-83 rain sensor with LM393 board | 1 | Analog + digital output | ₹60 |
| | **Total (approx.)** | | | **₹770** |

*Table 1: Bill of materials with approximate Indian prices (Robu.in / Amazon.in / local market, 2026)*

> Prices are indicative and vary by seller. The breadboard, cable and ESP32 can be reused across all ten projects, so the incremental cost of this project is about ₹210.

## 3.4 ESP32 DevKit V1

| Feature | Value |
|---|---|
| Module | ESP-WROOM-32 (ESP32-D0WD), dual-core Xtensa LX6 up to 240 MHz |
| Memory | 520 KB SRAM, 4 MB SPI flash |
| Wireless | Wi-Fi 802.11 b/g/n 2.4 GHz; Bluetooth 4.2 BR/EDR + BLE |
| I/O | 25 usable GPIO on the 30-pin board, 12-bit ADC (ADC1: GPIO32-39), 2 × DAC, PWM on any output |
| Interfaces | 3 × UART, 2 × I2C, 3 × SPI, I2S, touch, RMT |
| Supply | 5 V via USB/VIN, on-board 3.3 V regulator (AMS1117) |
| Logic level | 3.3 V — inputs are NOT 5 V tolerant |

*Table 2: Key specifications of the ESP32 DevKit V1*

## 3.5 Working Principle of Components

### DHT11

A capacitive humidity element and an NTC thermistor are read by a small on-board controller that sends a 40-bit frame (humidity, temperature, checksum) over a single-wire protocol. The ESP32 starts a reading by pulling the line low for 18 ms. The sensor allows one reading per second.

### BMP180

A piezo-resistive MEMS pressure element with an on-chip ADC and factory calibration coefficients stored in EEPROM. The Adafruit library reads the coefficients over I2C and applies Bosch's compensation formula to give pressure in pascals; the firmware converts it to hPa.

### LDR

Cadmium-sulphide resistance falls from about 1 MΩ in darkness to a few kΩ in bright light. With the LDR on the high side of a 10 kΩ divider, more light raises the voltage at GPIO34, which the 12-bit ADC reads in millivolts and scales to 0-100 %.

### FC-37 rain sensor

Interleaved copper tracks act as a variable resistor: water bridges the tracks and lowers the resistance. The LM393 board gives an analog voltage (lower = wetter) and a digital output that goes LOW above a threshold set by its potentiometer.

# 4. Circuit Design

## 4.1 Circuit Diagram

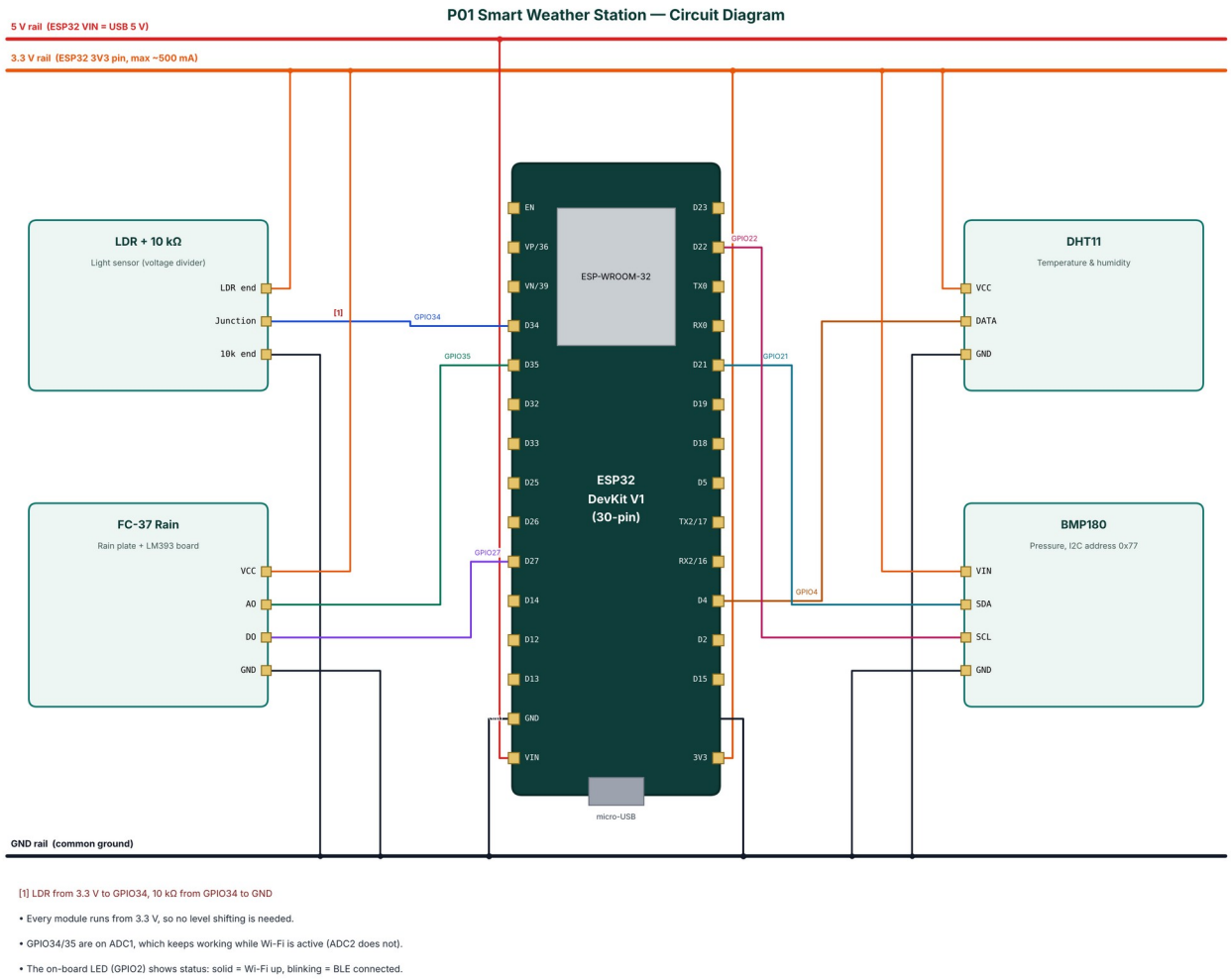



*Figure 2: Circuit diagram of the Smart Weather Station*

## 4.2 Pin Connections

| Module | Module pin | ESP32 connection | Notes |
|---|---|---|---|
| LDR + 10 kΩ | LDR end | 3.3 V (3V3 pin) | |
| LDR + 10 kΩ | Junction | GPIO34 | LDR from 3.3 V to GPIO34, 10 kΩ from GPIO34 to GND |
| LDR + 10 kΩ | 10k end | GND | |
| FC-37 Rain | VCC | 3.3 V (3V3 pin) | |
| FC-37 Rain | AO | GPIO35 | |
| FC-37 Rain | DO | GPIO27 | |
| FC-37 Rain | GND | GND | |
| DHT11 | VCC | 3.3 V (3V3 pin) | |
| DHT11 | DATA | GPIO4 | |
| DHT11 | GND | GND | |

| Module | Module pin | ESP32 connection | Notes |
|---|---|---|---|
| BMP180 | VIN | 3.3 V (3V3 pin) | |
| BMP180 | SDA | GPIO21 | |
| BMP180 | SCL | GPIO22 | |
| BMP180 | GND | GND | |

*Table 3: Pin connection table*

## 4.3 Interfacing and Power Notes

- Every module runs from 3.3 V, so no level shifting is needed.
- GPIO34/35 are on ADC1, which keeps working while Wi-Fi is active (ADC2 does not).
- The on-board LED (GPIO2) shows status: solid = Wi-Fi up, blinking = BLE connected.
- Only ADC1 pins (GPIO32-39) are used for analog inputs, because ADC2 cannot be read while Wi-Fi is active.
- Strapping pins GPIO0, 2, 12 and 15 are avoided for inputs so the board always boots normally.
- All grounds are common. 5 V signals are reduced to 3.3 V with resistor dividers before reaching the ESP32.

# 5. Data Flow Diagrams

Data flow diagrams (DFDs) show how data moves through the system without showing control logic or timing. External entities are drawn as rectangles, processes as numbered circles, data stores as open boxes and data flows as labelled arrows.

## 5.1 DFD Level 0 (Context Diagram)

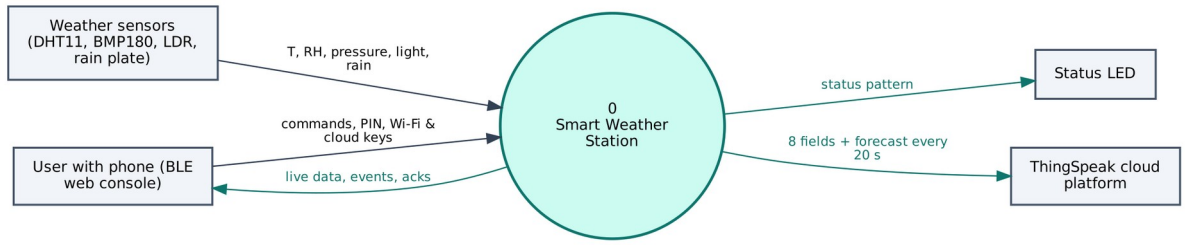



*Figure 3: DFD Level 0 — context diagram*

At level 0 the whole Smart Weather Station is a single process. It receives T, RH, pressure, light, rain from the Weather sensors (DHT11, BMP180, LDR, rain plate), commands and configuration from the user's phone, and no commands from the cloud. It sends status pattern to the Status LED, live data to the phone and 8 fields + forecast every 20 s to ThingSpeak.

## 5.2 DFD Level 1

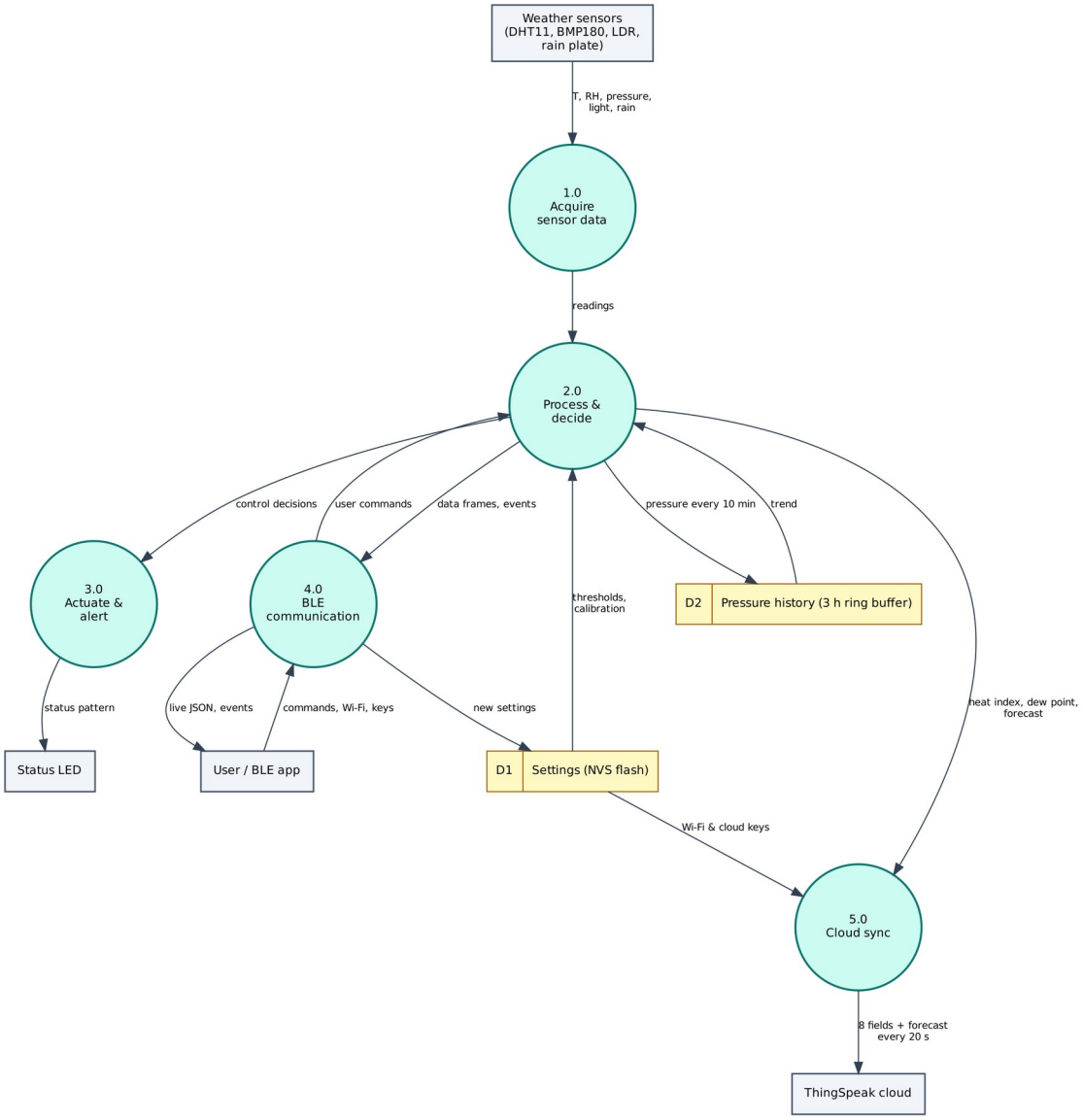



*Figure 4: DFD Level 1 — main processes and data stores*

| Process / store | Description |
|---|---|
| 1.0 Acquire sensor data | Reads T, RH, pressure, light, rain from the Weather sensors (DHT11, BMP180, LDR, rain plate); filters/averages the raw values. |
| 2.0 Process & decide | Computes heat index, dew point, forecast; applies thresholds and rules; generates events. |
| 3.0 Actuate & alert | Drives the Status LED (status pattern). |
| 4.0 BLE communication | Sends JSON frames and events to the phone; receives and authenticates commands. |
| 5.0 Cloud sync | Sends 8 fields + forecast every 20 s to ThingSpeak. |
| D1 Settings (NVS) | Wi-Fi credentials, cloud keys, PIN, thresholds — survive reboots. |
| D2 Pressure history (3 h ring buffer) | Stores pressure every 10 min; returns trend. |

*Table 4: Description of level-1 processes and data stores*

## 5.3 Data Dictionary

| Field | Meaning | Unit |
|---|---|---|
| t | Temperature | °C |
| h | Relative humidity | % |
| p | Pressure | hPa |
| l | Light level | % |
| rn | Rain wetness | % |
| hi | Heat index | °C |
| dp | Dew point | °C |
| fc | Forecast text | — |
| wifi | Wi-Fi connected | bool |

*Table 5: BLE data frame fields*

| Cloud object | Content / usage |
|---|---|
| Field 1 | Temperature (°C) |
| Field 2 | Humidity (%) |
| Field 3 | Pressure (hPa) |
| Field 4 | Light (%) |
| Field 5 | Rain (%) |
| Field 6 | Heat index (°C) |
| Field 7 | Dew point (°C) |
| Field 8 | Wi-Fi RSSI (dBm) |
| Status | Forecast text |

*Table 6: ThingSpeak data objects*

# 6. Software Design and Implementation

## 6.1 Development Environment

| Item | Details |
|---|---|
| IDE / build | Arduino IDE 2.x or arduino-cli; GitHub Actions builds firmware in the cloud |
| Board package | esp32 by Espressif Systems 3.x (tested with 3.3.12) |
| Board settings | ESP32 Dev Module, Partition Scheme: Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS) |
| Common libraries | NimBLE-Arduino 2.x, ArduinoJson 7.x, PubSubClient 2.8 |
| Project libraries | DHT sensor library (Adafruit), Adafruit Unified Sensor, Adafruit BMP085 Library |
| Phone app | Web Bluetooth console (Chrome on Android / desktop, Bluefy on iPhone); nRF Connect as alternative |
| Cloud | ThingSpeak (free tier) |

*Table 7: Software tools*

> The default 1.2 MB app partition is too small for BLE + Wi-Fi + TLS. Select **Minimal SPIFFS (1.9MB APP)** in Tools → Partition Scheme, otherwise the upload fails with 'Sketch too big'.

## 6.2 Firmware Architecture

The firmware is split into three layers so that the ten projects share tested code:

- **IoTCore.h** (shared) — Wi-Fi station with auto-reconnect, NimBLE GATT server with the Nordic UART Service, line-based command parser shared by BLE and USB serial, PIN authentication, settings in NVS (Preferences), NTP time in IST, JSON helpers.
- **Cloud.h** (shared) — ThingSpeak HTTPS client, Adafruit IO MQTT/TLS client with a 28 messages/minute rate limiter, Supabase PostgREST client (insert, update, select).
- **P01_Smart_Weather_Station.ino** — sensor drivers, algorithms, automation rules and the project's own commands.

The main loop is non-blocking: every task is scheduled with millis() timers, BLE writes are queued from the BLE callback into a FreeRTOS queue and executed in loop(), so the BLE stack never waits for slow operations such as HTTPS requests.

## 6.3 Working of the System

After power-up the firmware starts BLE advertising as IOT-P01-xxxx and tries the saved Wi-Fi network. Every two seconds it reads all sensors, averages 16 ADC samples for the analog channels and calculates heat index (Rothfusz regression, through the DHT library) and dew point using the Magnus formula: $\gamma = \ln(RH/100) + 17.62 \cdot T/(243.12+T)$, $Td = 243.12 \cdot \gamma/(17.62-\gamma)$.

Every 10 minutes the pressure is stored in a ring buffer of 18 samples (3 hours). The change between the oldest and newest sample, scaled to three hours, is classified: a fall of 3 hPa or more means storm or heavy rain is likely, a fall of 1.6 hPa means rain is likely, a steady value means stable weather and a rising value means improving weather. This is a simplified version of the rules used by barometric forecasters.

The live JSON frame is notified over BLE every two seconds. Every 20 seconds (configurable, minimum 15 s because of the ThingSpeak free-tier limit) the eight values are sent to ThingSpeak fields 1-8 with the

forecast as the status text. When the rain state changes, an event is sent over BLE and an immediate upload is scheduled.

## 6.4 Flowchart

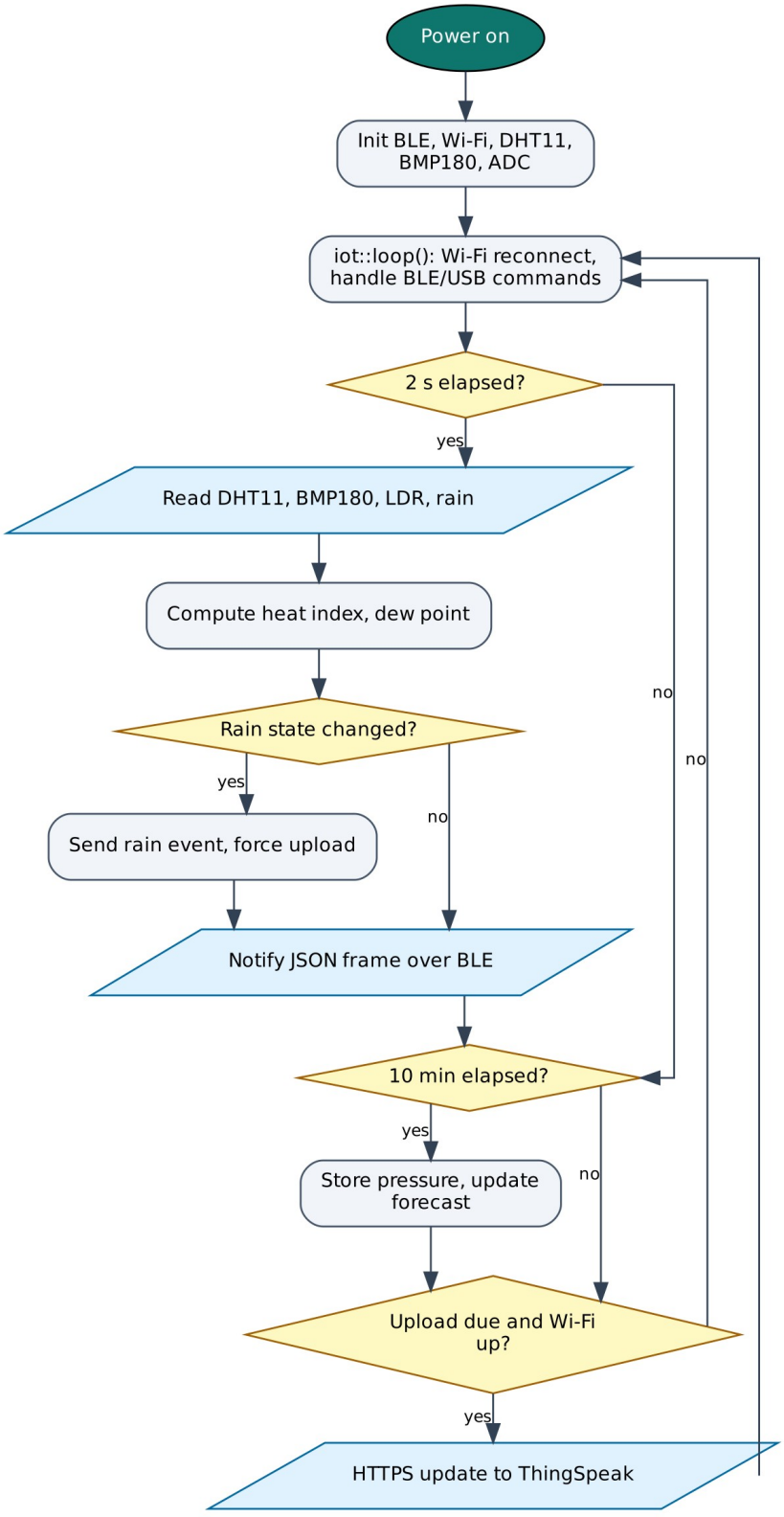



*Figure 5: Firmware flowchart*

## 6.5 BLE Communication Protocol

The device is a BLE peripheral named **IOT-P01-XXXX** (XXXX = last bytes of the MAC address). It exposes the Nordic UART Service, which most BLE apps understand:

| Item | UUID | Properties |
|---|---|---|
| Service (NUS) | 6E400001-B5A3-F393-E0A9-E50E24DCCA9E | Primary |
| RX — phone → ESP32 | 6E400002-B5A3-F393-E0A9-E50E24DCCA9E | Write, Write without response |
| TX — ESP32 → phone | 6E400003-B5A3-F393-E0A9-E50E24DCCA9E | Notify, Read |

*Table 8: GATT service and characteristics*

Commands are plain text lines (`command:argument`). Replies are one JSON object per line ending in a newline; long lines are split into MTU-sized notifications and re-joined by the app. Commands that change anything require `auth:<PIN>` first on each connection (default PIN 1234, change it with `set:pin=XXXX`).

| Command | PIN needed | Action |
|---|---|---|
| auth:<PIN> | — | Unlock for this connection |
| status | No | Status JSON: Wi-Fi, IP, RSSI, heap, uptime, time |
| wifi:<SSID>,<password> | Yes | Save Wi-Fi credentials and connect |
| set:<key>=<value> | Yes | Save a setting (cloud keys, thresholds) |
| get:<key> | Yes | Read a setting (secrets masked) |
| reboot / forget | Yes | Restart / erase all settings |
| read | No | Read the sensors now and send a data frame |
| upload | No | Upload to ThingSpeak as soon as the 15 s limit allows |

*Table 9: BLE commands (common commands first, then project commands)*

**Example data frame sent by the device:**

```
{"t":29.4,"h":62,"p":1004.6,"l":71,"rn":0,"hi":31.9,"dp":21.4,"fc":"Fair,
stable","wifi":true,"type":"data"}
```

## 6.6 Cloud Integration (ThingSpeak)

Data is sent with an HTTPS GET request: `https://api.thingspeak.com/update?api_key=KEY&field1=…&status=….` ThingSpeak returns the new entry number, or 0 if the update was rejected (for example when sent within 15 s of the previous one). The free licence allows 3 million messages per year and one update every 15 s per channel, so a 20 s interval leaves a safety margin.

| Object | Content |
|---|---|
| Field 1 | Temperature (°C) |
| Field 2 | Humidity (%) |
| Field 3 | Pressure (hPa) |
| Field 4 | Light (%) |
| Field 5 | Rain (%) |
| Field 6 | Heat index (°C) |
| Field 7 | Dew point (°C) |

| Object | Content |
|---|---|
| Field 8 | Wi-Fi RSSI (dBm) |
| Status | Forecast text |

*Table 10: ThingSpeak configuration*

## 6.7 Configurable Settings

| Key | Default | Meaning |
|---|---|---|
| tskey | — | ThingSpeak Write API Key |
| tsint | 20 | Upload interval in seconds (≥ 15) |
| rainthr | 30 | Rain threshold in % wetness |
| ssid / pass | — | Wi-Fi network (set with the wifi: command) |
| pin | 1234 | BLE PIN |

*Table 11: Settings stored in flash (change with set:key=value)*

# 7. Setup and Deployment Guide

## 7.1 Hardware Assembly

1. Place the ESP32 DevKit across the centre gap of the breadboard so both pin rows are free.
2. Connect the ESP32 3V3 pin to one breadboard power rail (3.3 V), VIN to the other rail (5 V) and both GND pins to the ground rails.
3. Wire each module exactly as in the pin connection table (section 4.2), including the resistor dividers marked with [n] in the circuit diagram.
4. Check every connection against the circuit diagram before powering on. Make sure no 5 V signal reaches an ESP32 pin directly.
5. Connect the USB cable to a computer (for flashing) or a 5 V adapter (for normal use).

## 7.2 Flashing the Firmware

### Option A — Arduino IDE

1. Install Arduino IDE 2.x from arduino.cc.
2. File → Preferences → Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`.
3. Tools → Board → Boards Manager: install **esp32 by Espressif Systems** (version 3.x).
4. Sketch → Include Library → Manage Libraries: install **NimBLE-Arduino** (2.x), **ArduinoJson** (7.x), **PubSubClient**, **DHT sensor library (Adafruit)**, **Adafruit Unified Sensor**, **Adafruit BMP085 Library**.
5. Download the repository and open the folder **P01_Smart_Weather_Station** (it already contains IoTCore.h, Cloud.h and config.h).
6. Tools: Board **ESP32 Dev Module**, Partition Scheme **Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)**, Upload Speed 921600, and the correct COM port.
7. Click Upload. If it stays at 'Connecting...', hold the BOOT button on the ESP32 until upload starts.
8. Open Serial Monitor at 115200 baud and type `status` to check the device.

### Option B — no software installation (GitHub Actions + browser flasher)

1. Open the repository https://github.com/rohitkukreja705/esp32-iot-projects → **Actions** → latest **Build firmware** run.
2. Download the artifact **firmware-P01_Smart_Weather_Station** and unzip it to get `P01_Smart_Weather_Station.ino.merged.bin`.
3. Open **https://espressif.github.io/esptool-js/** in Chrome or Edge on a computer, click **Connect** and select the ESP32's serial port.
4. Set Flash Address **0x0**, choose the merged .bin file and click **Program**. Press the EN (reset) button when done.

## 7.3 Cloud Setup (ThingSpeak)

1. Go to **thingspeak.mathworks.com** and sign up with a free MathWorks account (choose the free, non-commercial licence).

2. Open **Channels → My Channels → New Channel**. Name it 'Smart Weather Station'.
3. Tick and name the fields exactly as listed in the table below, then click **Save Channel**.
4. Open the **API Keys** tab and copy the **Write API Key** (16 characters).
5. In the BLE web console open **Setup**, unlock with the PIN and save the key in the `tskey` field (equivalent command: `set:tskey=YOURKEY`).
6. Open **Private View** to see live charts. Add Gauge or Numeric Display widgets with **Add Widgets**, and use **Sharing → Share channel view with everyone** if you want a public link.

| Field / feed / table | Content |
|---|---|
| Field 1 | Temperature (°C) |
| Field 2 | Humidity (%) |
| Field 3 | Pressure (hPa) |
| Field 4 | Light (%) |
| Field 5 | Rain (%) |
| Field 6 | Heat index (°C) |
| Field 7 | Dew point (°C) |
| Field 8 | Wi-Fi RSSI (dBm) |
| Status | Forecast text |

*Table 12: Items to create in ThingSpeak*

## 7.4 BLE App Setup (Web Bluetooth Console)

No app installation is needed. The console is a web page hosted free on GitHub Pages; Chrome talks to the ESP32 directly through the Web Bluetooth API.

1. On an Android phone turn on **Bluetooth** and **Location** (Android requires location for BLE scanning).
2. Open **https://rohitkukreja705.github.io/esp32-iot-projects/?p=P01** in **Chrome**. (On iPhone use the free **Bluefy** browser; on a laptop use Chrome or Edge.)
3. Tap **Connect** and choose **IOT-P01-XXXX** in the pop-up list. The dot turns green and the Live tab starts updating.
4. Open **Setup**, enter the PIN (default **1234**) and tap **Unlock**.
5. Enter your 2.4 GHz Wi-Fi name and password and tap **Save Wi-Fi**. After a few seconds the status line shows the IP address.
6. Enter the ThingSpeak keys from section 7.3 and tap **Save** next to each.
7. Use the **Control** tab for commands and the **Log** tab to see every event and acknowledgement.

### Alternative: nRF Connect app

1. Install **nRF Connect for Mobile** (Nordic Semiconductor, free) and scan.
2. Connect to **IOT-P01-XXXX** and open **Nordic UART Service**.
3. Enable notifications on the **TX** characteristic (6E400003…).
4. Write UTF-8 text to the **RX** characteristic (6E400002…), e.g. `auth:1234`, then `status`.

> To host the console yourself: in the GitHub repository open Settings → Pages, choose 'Deploy from a branch', branch main, folder /docs. Web Bluetooth only works on HTTPS pages, which GitHub Pages provides.

## 7.5 First-Run Checklist

- Serial Monitor shows '[BLE] advertising as IOT-P01-XXXX'.
- Status in the console shows wifi: true and an IP address.
- Data appears in ThingSpeak within a minute.
- All sensor values in the Live tab are plausible (no '—').

# 8. Testing and Results

The following test cases verify each function. Record the observed result and mark Pass/Fail while testing your build.

| # | Test | Procedure | Expected result | Observed |
|---|---|---|---|---|
| T1 | Sensor read | Open Serial Monitor at 115200 and send 'read' | Plausible T, RH, pressure (≈ 950-1020 hPa in most of India) | |
| T2 | Light response | Cover the LDR, then shine a phone torch on it | Light % drops below 20 then rises above 80 | |
| T3 | Rain detection | Put a few drops of water on the rain plate | Rain % rises, 'rain_start' event appears in the app log | |
| T4 | BLE provisioning | Connect from the web console, unlock with PIN, save Wi-Fi | Status shows Wi-Fi connected with an IP address | |
| T5 | Cloud upload | Wait 1 minute after Wi-Fi connects | New entries appear in the ThingSpeak channel charts | |
| T6 | Offline behaviour | Switch off the router | BLE data continues; uploads resume automatically when Wi-Fi returns | |
| T7 | Wrong PIN | Send auth:0000 then set:tsint=30 | Device replies 'wrong PIN' and refuses the setting | |

*Table 13: Test cases*

## 8.1 Expected Output

A typical data frame received in the console's Log tab looks like this (values will differ):

```
{"t":29.4,"h":62,"p":1004.6,"l":71,"rn":0,"hi":31.9,"dp":21.4,"fc":"Fair,
stable","wifi":true,"type":"data"}
```

Serial Monitor (115200 baud) prints the same lines along with connection messages such as `[WiFi] connected`, `[BLE] connected` and the HTTP or MQTT result of each cloud upload, which helps in troubleshooting.

## 8.2 Troubleshooting

| Symptom | Likely cause and fix |
|---|---|
| Device not in the Connect list | Bluetooth/Location off on the phone, or the page is not opened in Chrome over HTTPS. |
| 'Sketch too big' on upload | Select Partition Scheme 'Minimal SPIFFS (1.9MB APP with OTA)'. |
| Wi-Fi never connects | ESP32 supports only 2.4 GHz networks; check the SSID/password (case-sensitive). |
| Settings refused | Send auth:<PIN> first; the unlock is per BLE connection. |
| Sensor shows null / — | Check wiring and supply voltage; see the Serial Monitor for 'not found' messages. |
| Board resets randomly | Weak USB supply or servo/relay current; use a 5 V 2 A adapter and add a 470 µF capacitor. |
| No data in ThingSpeak | Wrong Write API Key or interval below 15 s. |

*Table 14: Troubleshooting guide*

# 9. Discussion

## 9.1 Advantages

- Very low cost; every part is easily available in India.
- Works locally over BLE even with no internet.
- No credentials in source code; configured from a phone.
- Free cloud storage with charts and a public view option.
- Gives a simple forecast, not just raw numbers.

## 9.2 Limitations

- DHT11 accuracy is ±2 °C / ±5 % RH; a DHT22 or BME280 gives better results.
- The resistive rain plate corrodes over months; a capacitive rain sensor lasts longer.
- The forecast uses pressure trend only and needs at least one hour of data.
- Wind and rainfall quantity (mm) are not measured.

## 9.3 Safety Precautions

- Keep the ESP32 and DHT11 inside a ventilated, rain-proof enclosure; expose only the rain plate.
- Power the rain plate only when reading it (or use a capacitive sensor) to slow corrosion.

## 9.4 Security Considerations

- Secrets (Wi-Fi password, cloud keys) are stored in the ESP32's NVS flash, not in source code.
- BLE commands that change the device require a PIN; change the default PIN after setup.
- Cloud traffic is TLS-encrypted. Certificate verification is disabled (setInsecure) for simplicity; for production, pin the server's root certificate with setCACert().
- Use a dedicated cloud account/project for the device and rotate keys if the device is lost.

## 9.5 Future Scope

- Add an anemometer and tipping-bucket rain gauge for wind and rainfall in mm.
- Replace DHT11 + BMP180 with a single BME280 for better accuracy.
- Run on a solar panel and Li-ion cell with deep sleep between readings.
- Use ThingSpeak MATLAB Analysis to send email alerts on heavy-rain prediction.

# 10. Conclusion

The Smart Weather Station shows that a useful, connected weather instrument can be built for under ₹1,000. By combining BLE and Wi-Fi it stays useful when the internet is unavailable, and the derived values and pressure-trend forecast make the data meaningful to non-technical users. The modular firmware and free cloud back-end make the system easy to extend with more sensors or a solar power supply.

All design goals listed in Chapter 1 are met with low-cost parts available in the Indian market, free cloud services and open-source software. The shared firmware core and the single Web Bluetooth console make the project easy to reproduce and extend.

# References


1. Aosong, DHT11 Humidity & Temperature Sensor datasheet.
2. Bosch Sensortec, BMP180 Digital Pressure Sensor datasheet (BST-BMP180-DS000).
3. Alduchov, O. A. & Eskridge, R. E. (1996), Improved Magnus form approximation of saturation vapor pressure, J. Applied Meteorology.
4. Rothfusz, L. P. (1990), The Heat Index Equation, NWS Technical Attachment SR 90-23.
5. Espressif Systems, ESP32 Series Datasheet and ESP32 Technical Reference Manual, espressif.com.
6. Espressif Systems, Arduino-ESP32 documentation, docs.espressif.com/projects/arduino-esp32.
7. H2zero, NimBLE-Arduino library documentation, github.com/h2zero/NimBLE-Arduino.
8. Bluetooth SIG, Bluetooth Core Specification 4.2 — Generic Attribute Profile (GATT).
9. W3C Web Bluetooth Community Group, Web Bluetooth specification, webbluetoothcg.github.io/web-bluetooth.
10. B. Blanchon, ArduinoJson 7 documentation, arduinojson.org.
11. MathWorks, ThingSpeak documentation — Write data with REST API, mathworks.com/help/thingspeak.

# Appendix A: Project Source Code

The complete code is in the repository folder **P01_Smart_Weather_Station** (https://github.com/rohitkukreja705/esp32-iot-projects).

### config.h

```
#pragma once
// ===== P01 Smart Weather Station — configuration =====
// Wi-Fi and cloud keys can stay empty here: send them from the BLE web app
// (Setup tab) and they are saved in the ESP32's flash.

#define WIFI_SSID     ""          // optional default
#define WIFI_PASS     ""
#define DEVICE_PIN    "1234"      // PIN the app must send before changing anything
#define TS_WRITE_KEY  ""          // ThingSpeak channel Write API Key (or set:tskey=...)

// ----- pins (ESP32 DevKit V1) -----
#define PIN_DHT       4           // DHT11 data
#define PIN_LDR       34          // LDR divider (ADC1)
#define PIN_RAIN_AO   35          // rain sensor analog out (ADC1)
#define PIN_RAIN_DO   27          // rain sensor digital out (LOW = wet)
#define PIN_LED       2           // on-board LED
// BMP180 uses I2C: SDA = GPIO21, SCL = GPIO22

// ----- timing -----
#define SAMPLE_MS          2000   // read sensors + BLE update
#define TS_INTERVAL_S      20     // ThingSpeak upload period (free tier minimum 15 s)
#define TREND_SAMPLE_MS    600000 // store pressure every 10 min for the 3-hour trend
```

### P01_Smart_Weather_Station.ino

```
/*
 * P01 — Smart Weather Station with Rain Forecast
 * ==============================================
 * Sensors : DHT11 (temperature, humidity), BMP180 (pressure),
 *           LDR (light), FC-37 rain sensor
 * Wi-Fi   : uploads 8 fields to ThingSpeak every 20 s (charts + public view)
 * BLE     : live readings every 2 s to the Web Bluetooth dashboard
 * Extras  : heat index, dew point, 3-hour pressure trend -> short-term forecast,
 *           rain-start / rain-stop events
 *
 * Board: ESP32 Dev Module | Partition: Minimal SPIFFS (1.9MB APP with OTA)
 * Libraries: NimBLE-Arduino 2.x, ArduinoJson 7, PubSubClient,
 *            DHT sensor library, Adafruit BMP085 Library
 */
#include "config.h"
#include "IoTCore.h"
#include "Cloud.h"
#include <Wire.h>
#include <DHT.h>
#include <Adafruit_BMP085.h>

DHT dht(PIN_DHT, DHT11);
Adafruit_BMP085 bmp;
bool bmpOK = false;

struct Reading {
  float temp = NAN, hum = NAN, pres = NAN, light = NAN, rain = NAN;
  float heatIndex = NAN, dewPoint = NAN;
  bool raining = false;
```

```
} r;

// pressure history for trend (18 samples x 10 min = 3 h)
float presHist[18]; int presCount = 0, presIdx = 0;
String forecast = "Collecting data";
uint32_t lastSample = 0, lastUpload = 0, lastTrend = 0;
uint32_t uploadPeriod = TS_INTERVAL_S * 1000UL;
bool forceUpload = false;

// Average several ADC readings (in millivolts) to reduce noise.
float readMilliVolts(int pin, int n = 16) {
  uint32_t s = 0;
  for (int i = 0; i < n; i++) { s += analogReadMilliVolts(pin); delayMicroseconds(200); }
  return s / (float)n;
}

// Magnus formula, valid -45..60 °C
float dewPointC(float t, float h) {
  if (isnan(t) || isnan(h) || h <= 0) return NAN;
  const float a = 17.62, b = 243.12;
  float g = log(h / 100.0) + a * t / (b + t);
  return b * g / (a - g);
}

// Very small Zambretti-style rule set based on 3-hour pressure change.
void updateForecast() {
  if (presCount < 6) { forecast = "Collecting data"; return; }   // need >= 1 h
  int oldest = (presIdx - presCount + 18) % 18;
  float trend = presHist[(presIdx - 1 + 18) % 18] - presHist[oldest];
  // scale to a 3-hour change
  trend = trend * 18.0 / presCount;
  float p = r.pres;
  if (trend <= -3.0)       forecast = "Storm / heavy rain likely";
  else if (trend <= -1.6)  forecast = "Rain likely";
  else if (trend < -0.5)   forecast = "Clouds increasing";
  else if (trend < 0.5)    forecast = (p > 1015) ? "Fair, stable" : "Unsettled, stable";
  else if (trend < 1.6)    forecast = "Improving";
  else                     forecast = "Clearing, fair weather";
}

void readSensors() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (!isnan(t)) r.temp = t;
  if (!isnan(h)) r.hum = h;
  if (bmpOK) r.pres = bmp.readPressure() / 100.0;          // Pa -> hPa
  // LDR on high side: more light -> higher voltage
  r.light = constrain(readMilliVolts(PIN_LDR) / 3300.0 * 100.0, 0, 100);
  // Rain plate: dry ~3300 mV, wet -> lower voltage
  float rainMv = readMilliVolts(PIN_RAIN_AO);
  r.rain = constrain((3300.0 - rainMv) / 3300.0 * 100.0, 0, 100);
  r.heatIndex = (isnan(r.temp) || isnan(r.hum)) ? NAN : dht.computeHeatIndex(r.temp, r.hum, false);
  r.dewPoint = dewPointC(r.temp, r.hum);

  bool wet = digitalRead(PIN_RAIN_DO) == LOW || r.rain > iot::settingF("rainthr", 30);
  if (wet != r.raining) {
    r.raining = wet;
    iot::sendEvent(wet ? "rain_start" : "rain_stop", wet ? "Rain detected" : "Rain stopped");
    forceUpload = true;
  }
}

void sendBle() {
```

```
  JsonDocument d;
  d["t"]  = iot::rnd(r.temp);
  d["h"]  = iot::rnd(r.hum, 0);
  d["p"]  = iot::rnd(r.pres);
  d["l"]  = iot::rnd(r.light, 0);
  d["rn"] = iot::rnd(r.rain, 0);
  d["hi"] = iot::rnd(r.heatIndex);
  d["dp"] = iot::rnd(r.dewPoint);
  d["fc"] = forecast;
  d["wifi"] = iot::wifiOK();
  iot::sendData(d);
}

void upload() {
  float f[8] = { r.temp, r.hum, r.pres, r.light, r.rain, r.heatIndex, r.dewPoint,
                 (float)WiFi.RSSI() };
  bool ok = cloud::thingSpeakUpdate(f, 8, forecast);
  if (!ok) iot::sendEvent("upload_failed", "ThingSpeak upload failed");
}

// Project-specific commands (everything else is handled by IoTCore)
bool onCommand(const String& cmd, const String& arg, bool authed) {
  if (cmd == "read")   { readSensors(); sendBle(); return true; }
  if (cmd == "upload") { forceUpload = true; iot::emitAck(cmd, true, "queued"); return true; }
  if (cmd == "setting") return true;   // a threshold changed: read live via settingF()
  return false;
}

void setup() {
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_RAIN_DO, INPUT);
  analogSetAttenuation(ADC_11db);           // full 0-3.3 V range
  iot::begin("P01", onCommand);
  dht.begin();
  Wire.begin(21, 22);
  bmpOK = bmp.begin();
  if (!bmpOK) Serial.println("[BMP180] not found - check SDA/SCL wiring");
}

void loop() {
  iot::loop();
  uint32_t now = millis();

  if (now - lastSample >= SAMPLE_MS) {
    lastSample = now;
    readSensors();
    sendBle();
    uploadPeriod = max(15, (int)iot::settingF("tsint", TS_INTERVAL_S)) * 1000UL;
  }
  if (bmpOK && (now - lastTrend >= TREND_SAMPLE_MS || presCount == 0) && !isnan(r.pres)) {
    lastTrend = now;
    presHist[presIdx] = r.pres;
    presIdx = (presIdx + 1) % 18;
    if (presCount < 18) presCount++;
    updateForecast();
  }
  if ((now - lastUpload >= uploadPeriod || forceUpload) && iot::wifiOK() && now - lastUpload > 15500) {
    lastUpload = now;
    forceUpload = false;
    upload();
  }
  // LED: solid = Wi-Fi up, slow blink = BLE connected, off = offline
  digitalWrite(PIN_LED, iot::wifiOK() ? HIGH : (iot::bleConnected() ? (now / 500) % 2 : LOW));
```

```
}
```

# Appendix B: Shared Library Code

These two headers are identical in every project folder (master copies in /common).

**IoTCore.h**

```
/*
 * IoTCore.h  —  shared runtime for all ESP32 IoT projects in this repo
 * ---------------------------------------------------------------------
 *  - Wi-Fi station with automatic reconnect (credentials stored in flash)
 *  - BLE server using the Nordic UART Service (NUS) layout, so the device
 *    works with the Web Bluetooth dashboard in /docs AND with generic apps
 *    such as "nRF Connect" or "Serial Bluetooth Terminal"
 *  - Line-based command protocol shared by BLE and USB Serial
 *  - Persistent settings (Preferences / NVS): Wi-Fi, cloud keys, PIN, thresholds
 *  - NTP time sync (IST) for timestamps
 *
 *  Wire protocol
 *  -------------
 *  Phone -> ESP32 (write to RX characteristic, text):
 *      auth:1234              unlock control + settings for this connection
 *      status                 device replies with a status JSON line
 *      set:key=value          store a setting (Wi-Fi, cloud key, threshold…)
 *      get:key                read back a setting (secrets are masked)
 *      wifi:SSID,PASSWORD     shortcut for set:ssid + set:pass + reconnect
 *      reboot                 restart the board
 *      <anything else>        forwarded to the project's own command handler
 *  ESP32 -> Phone (notify on TX characteristic):
 *      one JSON object per line, terminated by '\n'
 *      {"type":"data",...}  {"type":"status",...}  {"type":"ack",...}
 *      {"type":"event",...} {"type":"err",...}
 *  Lines longer than one BLE packet are split into chunks; the receiver
 *  joins chunks until it sees '\n'.
 *
 *  Library: NimBLE-Arduino 2.x (lighter than Bluedroid, leaves more RAM for TLS)
 *  Board:   ESP32 Dev Module, Partition Scheme "Minimal SPIFFS (1.9MB APP with OTA)"
 */
#pragma once

#include <Arduino.h>
#include <WiFi.h>
#include <Preferences.h>
#include <NimBLEDevice.h>
#include <ArduinoJson.h>
#include <time.h>

#ifndef DEVICE_PIN
#define DEVICE_PIN "1234"
#endif
#ifndef WIFI_SSID
#define WIFI_SSID ""
#endif
#ifndef WIFI_PASS
#define WIFI_PASS ""
#endif

namespace iot {

// Nordic UART Service UUIDs (widely supported by BLE terminal apps)
static const char* NUS_SERVICE = "6E400001-B5A3-F393-E0A9-E50E24DCCA9E";
static const char* NUS_RX      = "6E400002-B5A3-F393-E0A9-E50E24DCCA9E";  // phone writes here
static const char* NUS_TX      = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E";  // ESP32 notifies here
```

```
// Project command handler: return true if the command was recognised.
using CommandHandler = bool (*)(const String& cmd, const String& arg, bool authed);

// ---------------------------------------------------------------- state
struct CmdMsg { char text[256]; bool fromBle; };

static Preferences        prefs;
static NimBLEServer*      bleServer   = nullptr;
static NimBLECharacteristic* txChr    = nullptr;
static QueueHandle_t      cmdQueue    = nullptr;
static CommandHandler     userHandler = nullptr;
static String             deviceName;
static String             projectId;
static volatile uint16_t  bleMtu      = 23;
static volatile bool      bleAuthed   = false;   // reset on every new connection
static bool               serialAuthed = true;   // USB = physical access, trusted
static bool               timeSynced  = false;
static uint32_t           lastWifiTry = 0;
static String             rxLine;                // assembles multi-packet writes

// ---------------------------------------------------------------- settings
inline String setting(const char* key, const char* def = "") {
  return prefs.isKey(key) ? prefs.getString(key, def) : String(def);
}
inline void saveSetting(const char* key, const String& v) { prefs.putString(key, v); }
inline float settingF(const char* key, float def) {
  String s = setting(key, "");
  return s.length() ? s.toFloat() : def;
}

// ---------------------------------------------------------------- BLE
inline bool bleConnected() { return bleServer && bleServer->getConnectedCount() > 0; }

// Send one line (adds '\n'), split to fit the negotiated MTU.
inline void bleSend(const String& line) {
  if (!bleConnected() || !txChr) return;
  String msg = line + "\n";
  size_t chunk = (bleMtu > 23 ? bleMtu : 23) - 3;
  for (size_t i = 0; i < msg.length(); i += chunk) {
    size_t n = min(chunk, msg.length() - i);
    txChr->setValue((const uint8_t*)msg.c_str() + i, n);
    txChr->notify();
    if (msg.length() > chunk) delay(8);   // give the stack time between chunks
  }
}

// Send to BLE and mirror to USB Serial.
inline void emit(const String& line) {
  Serial.println(line);
  bleSend(line);
}

inline void emitAck(const String& cmd, bool ok, const String& msg = "") {
  JsonDocument d;
  d["type"] = ok ? "ack" : "err";
  d["cmd"] = cmd;
  if (msg.length()) d["msg"] = msg;
  String s; serializeJson(d, s); emit(s);
}

inline void enqueue(const String& s, bool fromBle) {
  if (!cmdQueue || s.length() == 0) return;
```

```
  CmdMsg m; memset(&m, 0, sizeof(m));
  strncpy(m.text, s.c_str(), sizeof(m.text) - 1);
  m.fromBle = fromBle;
  xQueueSend(cmdQueue, &m, 0);
}

class ServerCB : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer* s, NimBLEConnInfo& info) override {
    bleAuthed = false;
    rxLine = "";
    Serial.printf("[BLE] connected %s\n", info.getAddress().toString().c_str());
  }
  void onDisconnect(NimBLEServer* s, NimBLEConnInfo& info, int reason) override {
    bleAuthed = false;
    bleMtu = 23;
    Serial.printf("[BLE] disconnected (reason %d), advertising again\n", reason);
    // NimBLE restarts advertising automatically (advertiseOnDisconnect = true)
  }
  void onMTUChange(uint16_t mtu, NimBLEConnInfo& info) override {
    bleMtu = mtu;
    Serial.printf("[BLE] MTU %u\n", mtu);
  }
};

class RxCB : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c, NimBLEConnInfo& info) override {
    std::string v = c->getValue();
    // Accept either a single write without newline, or several writes
    // that together form lines terminated by '\n'.
    for (char ch : v) {
      if (ch == '\r') continue;
      if (ch == '\n') { enqueue(rxLine, true); rxLine = ""; }
      else if (rxLine.length() < 250) rxLine += ch;
    }
    if (v.find('\n') == std::string::npos && rxLine.length() && rxLine.length() == v.length()) {
      enqueue(rxLine, true);  // plain one-shot write (e.g. typed in nRF Connect)
      rxLine = "";
    }
  }
};

inline void bleBegin() {
  uint8_t mac[6]; WiFi.macAddress(mac);
  char suffix[6]; snprintf(suffix, sizeof(suffix), "%02X%02X", mac[4], mac[5]);
  deviceName = "IOT-" + projectId + "-" + suffix;

  NimBLEDevice::init(deviceName.c_str());
  NimBLEDevice::setMTU(247);
  bleServer = NimBLEDevice::createServer();
  bleServer->setCallbacks(new ServerCB());

  NimBLEService* svc = bleServer->createService(NUS_SERVICE);
  txChr = svc->createCharacteristic(NUS_TX, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ);
  NimBLECharacteristic* rx = svc->createCharacteristic(
      NUS_RX, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR, 512);
  rx->setCallbacks(new RxCB());
  // (NimBLE 2.x starts services automatically when advertising begins)

  // 128-bit UUID goes in the advertisement, the name in the scan response
  // (both together would not fit in one 31-byte packet).
  NimBLEAdvertisementData adv, scan;
  adv.setFlags(BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP);
  adv.setCompleteServices(NimBLEUUID(NUS_SERVICE));
```

```
  scan.setName(deviceName.c_str());
  NimBLEAdvertising* a = NimBLEDevice::getAdvertising();
  a->setAdvertisementData(adv);
  a->setScanResponseData(scan);
  a->start();
  Serial.printf("[BLE] advertising as %s\n", deviceName.c_str());
}

// ---------------------------------------------------------------- Wi-Fi
inline bool wifiOK() { return WiFi.status() == WL_CONNECTED; }

inline void wifiConnect() {
  String ssid = setting("ssid", WIFI_SSID);
  String pass = setting("pass", WIFI_PASS);
  if (ssid.isEmpty()) {
    Serial.println("[WiFi] no SSID yet - send  wifi:SSID,PASSWORD  over BLE");
    return;
  }
  Serial.printf("[WiFi] connecting to %s\n", ssid.c_str());
  WiFi.disconnect();
  WiFi.begin(ssid.c_str(), pass.c_str());
  lastWifiTry = millis();
}

inline void wifiLoop() {
  static bool wasUp = false;
  bool up = wifiOK();
  if (up && !wasUp) {
    Serial.printf("[WiFi] connected, IP %s, RSSI %d\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
    configTzTime("IST-5:30", "pool.ntp.org", "time.google.com");
  }
  if (!up && millis() - lastWifiTry > 20000) wifiConnect();   // retry every 20 s
  wasUp = up;
  if (up && !timeSynced && time(nullptr) > 1700000000) {
    timeSynced = true;
    Serial.println("[NTP] time synced");
  }
}

// ISO-8601 local time, or uptime if NTP has not synced yet.
inline String timestamp() {
  time_t now = time(nullptr);
  if (now < 1700000000) return "uptime+" + String(millis() / 1000) + "s";
  struct tm t; localtime_r(&now, &t);
  char buf[32]; strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S+05:30", &t);
  return String(buf);
}

// ------------------------------------------------------- built-in commands
inline bool isSecretKey(const String& k) {
  return k == "pass" || k.endsWith("key") || k == "pin";
}

inline void sendStatus() {
  JsonDocument d;
  d["type"] = "status";
  d["dev"] = deviceName;
  d["wifi"] = wifiOK();
  d["ssid"] = setting("ssid", WIFI_SSID);
  d["rssi"] = wifiOK() ? WiFi.RSSI() : 0;
  d["ip"] = wifiOK() ? WiFi.localIP().toString() : "";
  d["auth"] = bleAuthed;
```

```
  d["heap"] = ESP.getFreeHeap();
  d["up"] = millis() / 1000;
  d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

inline void handleLine(const String& raw, bool fromBle) {
  String line = raw; line.trim();
  if (line.isEmpty()) return;
  int c = line.indexOf(':');
  String cmd = (c < 0 ? line : line.substring(0, c));
  String arg = (c < 0 ? "" : line.substring(c + 1));
  cmd.trim(); cmd.toLowerCase();
  bool authed = fromBle ? bleAuthed : serialAuthed;

  if (cmd == "auth") {
    bool ok = arg == setting("pin", DEVICE_PIN);
    if (fromBle) bleAuthed = ok;
    emitAck(cmd, ok, ok ? "unlocked" : "wrong PIN");
    return;
  }
  if (cmd == "status") { sendStatus(); return; }
  if (cmd == "ping")   { emitAck(cmd, true, "pong"); return; }

  // Everything below changes the device, so it needs the PIN over BLE.
  if (cmd == "set" || cmd == "wifi" || cmd == "reboot" || cmd == "get" || cmd == "forget") {
    if (!authed) { emitAck(cmd, false, "send auth:<PIN> first"); return; }
    if (cmd == "set") {
      int e = arg.indexOf('=');
      if (e <= 0) { emitAck(cmd, false, "use set:key=value"); return; }
      String k = arg.substring(0, e); k.trim();
      String v = arg.substring(e + 1); v.trim();
      if (k.length() > 15) { emitAck(cmd, false, "key too long"); return; }
      saveSetting(k.c_str(), v);
      emitAck(cmd, true, k + " saved");
      if (k == "ssid" || k == "pass") wifiConnect();
      // let the project react to changed thresholds etc.
      if (userHandler) userHandler("setting", k, true);
      return;
    }
    if (cmd == "get") {
      String v = setting(arg.c_str(), "");
      if (isSecretKey(arg) && v.length()) v = "****" + v.substring(max(0, (int)v.length() - 3));
      emitAck(cmd, true, arg + "=" + v);
      return;
    }
    if (cmd == "wifi") {
      int k = arg.indexOf(',');
      String ssid = k < 0 ? arg : arg.substring(0, k);
      String pass = k < 0 ? "" : arg.substring(k + 1);
      saveSetting("ssid", ssid); saveSetting("pass", pass);
      emitAck(cmd, true, "connecting to " + ssid);
      wifiConnect();
      return;
    }
    if (cmd == "forget") { prefs.clear(); emitAck(cmd, true, "all settings erased"); return; }
    if (cmd == "reboot") { emitAck(cmd, true, "rebooting"); delay(300); ESP.restart(); }
  }

  if (userHandler && userHandler(cmd, arg, authed)) return;
  emitAck(cmd, false, "unknown command");
}
```

```
// ------------------------------------------------------------ entry points
// Call once from setup().  id = short project id, e.g. "P01".
inline void begin(const char* id, CommandHandler handler) {
  Serial.begin(115200);
  delay(200);
  projectId = id;
  userHandler = handler;
  prefs.begin("iot", false);
  cmdQueue = xQueueCreate(8, sizeof(CmdMsg));
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  // NOTE: Wi-Fi modem sleep must stay ON while BLE is running (ESP32 coexistence).
  bleBegin();
  wifiConnect();
  Serial.println("[CORE] ready - type 'status' here or connect over BLE");
}

// Call every loop() iteration.
inline void loop() {
  wifiLoop();
  // USB serial commands (handy while developing)
  static String serialLine;
  while (Serial.available()) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') { if (serialLine.length()) enqueue(serialLine, false); serialLine =
""; }
    else if (serialLine.length() < 250) serialLine += ch;
  }
  CmdMsg m;
  while (cmdQueue && xQueueReceive(cmdQueue, &m, 0) == pdTRUE) handleLine(String(m.text), m.fromBle);
}

// Round for compact JSON. NaN stays NaN, which ArduinoJson writes as null.
inline float rnd(float v, int dp = 1) {
  if (isnan(v)) return v;
  float m = powf(10, dp);
  return roundf(v * m) / m;
}

// Helper: build and send a {"type":"data",...} line from a filled document.
inline void sendData(JsonDocument& d) {
  d["type"] = "data";
  String s; serializeJson(d, s); emit(s);
}

inline void sendEvent(const String& name, const String& msg) {
  JsonDocument d;
  d["type"] = "event"; d["name"] = name; d["msg"] = msg; d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

}  // namespace iot
```

### Cloud.h

```
/*
 * Cloud.h  –  free-tier cloud helpers used by the projects
 * ---------------------------------------------------------
 *  ThingSpeak   : HTTPS REST "update" (free tier: 1 update / 15 s per channel)
 *  Adafruit IO  : MQTT over TLS (free tier: 30 data points / minute, 10 feeds)
 *  Supabase     : PostgREST over HTTPS (free tier: 500 MB Postgres)
 *
```

```
 *  Keys are read from settings (stored over BLE with set:key=value) so no
 *  secret has to be hard-coded or committed to GitHub:
 *     tskey    ThingSpeak Write API Key
 *     aiouser  Adafruit IO username        aiokey  Adafruit IO key
 *     sburl    https://<ref>.supabase.co   sbkey   Supabase publishable/anon key
 *
 *  TLS note: certificate checking is disabled with setInsecure() to keep the
 *  projects beginner-friendly and independent of CA rotations. Traffic is
 *  still encrypted. For production, pin the root CA with setCACert().
 */
#pragma once

#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include "IoTCore.h"

// Optional compile-time defaults (normally left empty and set over BLE)
#ifndef TS_WRITE_KEY
#define TS_WRITE_KEY ""
#endif
#ifndef AIO_USERNAME
#define AIO_USERNAME ""
#endif
#ifndef AIO_KEY
#define AIO_KEY ""
#endif
#ifndef SUPABASE_URL
#define SUPABASE_URL ""
#endif
#ifndef SUPABASE_KEY
#define SUPABASE_KEY ""
#endif

namespace cloud {

// ============================================================== ThingSpeak
// vals[i] -> field(i+1). Pass NAN to skip a field. Returns true on success.
inline bool thingSpeakUpdate(const float* vals, int n, const String& status = "") {
  if (!iot::wifiOK()) return false;
  String key = iot::setting("tskey", TS_WRITE_KEY);
  if (key.isEmpty()) { Serial.println("[TS] no tskey set"); return false; }
  String url = "https://api.thingspeak.com/update?api_key=" + key;
  for (int i = 0; i < n && i < 8; i++)
    if (!isnan(vals[i])) url += "&field" + String(i + 1) + "=" + String(vals[i], 2);
  if (status.length()) {
    String st = status; st.replace(" ", "%20");
    url += "&status=" + st;
  }
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, url)) return false;
  int code = http.GET();
  String body = http.getString();
  http.end();
  bool ok = code == 200 && body.toInt() > 0;   // ThingSpeak returns the entry id, 0 = rejected
  Serial.printf("[TS] HTTP %d entry=%s\n", code, body.c_str());
  return ok;
}

// ============================================================== Supabase
// Generic PostgREST call. method: GET/POST/PATCH. path e.g. "/rest/v1/readings"
inline int supabaseRequest(const char* method, const String& path, const String& body,
```

```
                           String* response = nullptr, const char* prefer = "return=minimal") {
  if (!iot::wifiOK()) return -1;
  String base = iot::setting("sburl", SUPABASE_URL);
  String key  = iot::setting("sbkey", SUPABASE_KEY);
  if (base.isEmpty() || key.isEmpty()) { Serial.println("[SB] sburl/sbkey not set"); return -2; }
  if (base.endsWith("/")) base.remove(base.length() - 1);
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, base + path)) return -3;
  http.addHeader("apikey", key);
  http.addHeader("Authorization", "Bearer " + key);
  http.addHeader("Content-Type", "application/json");
  if (prefer) http.addHeader("Prefer", prefer);
  int code = http.sendRequest(method, body);
  if (response) *response = http.getString();
  else if (code >= 300) Serial.printf("[SB] %s\n", http.getString().c_str());
  http.end();
  Serial.printf("[SB] %s %s -> %d\n", method, path.c_str(), code);
  return code;
}

inline bool supabaseInsert(const char* table, const String& json) {
  int c = supabaseRequest("POST", String("/rest/v1/") + table, json);
  return c >= 200 && c < 300;
}

// filter is a PostgREST filter, e.g. "id=eq.1"
inline bool supabaseUpdate(const char* table, const String& filter, const String& json) {
  int c = supabaseRequest("PATCH", String("/rest/v1/") + table + "?" + filter, json);
  return c >= 200 && c < 300;
}

// query e.g. "select=*&id=eq.1"  -> JSON array text in out
inline bool supabaseSelect(const char* table, const String& query, String& out) {
  int c = supabaseRequest("GET", String("/rest/v1/") + table + "?" + query, "", &out, nullptr);
  return c == 200;
}

// ============================================================ Adafruit IO
// MQTT wrapper with a simple rate limiter to respect the 30 msg/min limit.
using FeedCallback = void (*)(const String& feed, const String& value);

class AdafruitIO {
 public:
  // requestLast = true asks Adafruit IO for each feed's last value on connect
  // (useful for switches changed while offline; unsafe for things like pumps)
  void begin(FeedCallback cb, const char* const* subFeeds, int nSub, bool requestLast = true) {
    cb_ = cb; subs_ = subFeeds; nSub_ = nSub; requestLast_ = requestLast;
    tls_.setInsecure();
    mqtt_.setClient(tls_);
    mqtt_.setServer("io.adafruit.com", 8883);
    mqtt_.setBufferSize(512);
    mqtt_.setKeepAlive(60);
    mqtt_.setCallback([](char* topic, byte* payload, unsigned int len) {
      String t(topic), v;
      for (unsigned int i = 0; i < len; i++) v += (char)payload[i];
      int p = t.lastIndexOf('/');
      String feed = t.substring(p + 1);
      if (instance()->cb_) instance()->cb_(feed, v);
    });
    instance() = this;
  }
```

```
  void loop() {
    if (!iot::wifiOK()) return;
    if (!mqtt_.connected()) {
      if (millis() - lastTry_ < 10000) return;
      lastTry_ = millis();
      user_ = iot::setting("aiouser", AIO_USERNAME);
      String key = iot::setting("aiokey", AIO_KEY);
      if (user_.isEmpty() || key.isEmpty()) { Serial.println("[AIO] aiouser/aiokey not set"); return; }
      String id = "esp32-" + String((uint32_t)ESP.getEfuseMac(), HEX);
      Serial.println("[AIO] connecting...");
      if (mqtt_.connect(id.c_str(), user_.c_str(), key.c_str())) {
        Serial.println("[AIO] connected");
        for (int i = 0; i < nSub_; i++) {
          mqtt_.subscribe(topic(subs_[i]).c_str());
          if (requestLast_) mqtt_.publish((topic(subs_[i]) + "/get").c_str(), "");
        }
        // subscribe to throttle/error topics so we learn about rate limits
        mqtt_.subscribe((user_ + "/throttle").c_str());
      } else {
        Serial.printf("[AIO] failed, state %d\n", mqtt_.state());
      }
    }
    mqtt_.loop();
  }

  bool connected() { return mqtt_.connected(); }

  // Returns false if not connected or if the per-minute budget is used up.
  bool publish(const char* feed, const String& value) {
    if (!mqtt_.connected()) return false;
    uint32_t now = millis();
    if (now - windowStart_ > 60000) { windowStart_ = now; sent_ = 0; }
    if (sent_ >= 28) { Serial.println("[AIO] rate budget used, skipping"); return false; }
    sent_++;
    return mqtt_.publish(topic(feed).c_str(), value.c_str());
  }
  bool publish(const char* feed, float v, int dp = 2) { return publish(feed, String(v, dp)); }

 private:
  String topic(const char* feed) { return user_ + "/feeds/" + feed; }
  static AdafruitIO*& instance() { static AdafruitIO* p = nullptr; return p; }

  WiFiClientSecure tls_;
  PubSubClient mqtt_;
  FeedCallback cb_ = nullptr;
  const char* const* subs_ = nullptr;
  int nSub_ = 0;
  bool requestLast_ = true;
  String user_;
  uint32_t lastTry_ = 0, windowStart_ = 0;
  int sent_ = 0;
};

}  // namespace cloud
```